

Requerimientos y Reglas de Negocio

1. Requerimientos Funcionales

1.1 Autenticación y Usuarios

- El sistema debe permitir registro de usuarios con username, email y contraseña.
- El registro es un proceso de dos pasos: primero se validan los datos y se envía un código de verificación de 6 dígitos al email del usuario; el usuario solo se crea en la base de datos al verificar el código.
- El código de verificación expira a los 15 minutos. El usuario puede solicitar el reenvío del código al mismo email.
- El sistema debe permitir inicio y cierre de sesión. La sesión se gestiona mediante JWT con expiración de 7 días.
- El sistema debe permitir solicitar el restablecimiento de contraseña mediante un enlace enviado al email del usuario. El enlace expira en 1 hora.
- El usuario puede ver y editar su perfil (username, bio, foto de perfil).
- El perfil público muestra los puntos acumulados, laboratorios completados y posición en el ranking.

1.2 Cursos y Contenido

- El sistema debe listar todos los cursos publicados con su título, descripción, dificultad, cantidad de módulos, laboratorios y puntos totales.
- La lista de cursos indica si el usuario autenticado ya está inscrito en cada curso.
- El usuario puede inscribirse a un curso. Solo puede inscribirse una vez por curso.
- Cada curso contiene módulos ordenados por posición.
- Cada módulo contiene laboratorios ordenados por posición.
- El usuario solo puede acceder al contenido completo de un laboratorio (incluyendo preguntas) si está inscrito en su curso.
- El contenido de un curso, módulo o laboratorio despublicado no es accesible para usuarios regulares, aunque estén inscritos.

1.3 Laboratorios y Preguntas

- Cada laboratorio muestra su contenido en Markdown antes de comenzar el quiz.
- El quiz de cada laboratorio tiene exactamente 5 preguntas.
- Las preguntas pueden ser de tipo selección múltiple (`multiple_choice`) o de tipo actividad (`activity_response`).
- El usuario debe responder las 5 preguntas para poder enviar el laboratorio.
- El sistema calcula automáticamente el puntaje al enviar. El frontend nunca envía el puntaje calculado.
- Después de enviar, el sistema muestra retroalimentación por pregunta, incluyendo la explicación de cada una.
- Cada laboratorio tiene un tiempo estimado de completación (`estimated_minutes`) y una cantidad de puntos asociada.

1.4 Actividades Prácticas

- Cada pregunta de tipo actividad muestra instrucciones y la acción a realizar.
- Al ejecutar la acción correcta, el sistema genera una respuesta única por usuario mediante un hash determinístico (HMAC-SHA256 de la combinación `userId` + `activityId`, truncado a 16 caracteres en mayúsculas).
- El hash generado es siempre el mismo para la misma combinación de usuario y actividad, de modo que el usuario puede recuperarlo al reintentar.
- El usuario debe copiar la respuesta generada e ingresarla en la pregunta correspondiente para que cuente como correcta.
- El sistema registra cada intento de actividad en un log inmutable (`activity_action_logs`), indicando si fue correcto o no.
- El usuario debe completar exitosamente la actividad antes de poder responder correctamente su pregunta asociada, ya que la respuesta correcta se genera en ese momento.

1.5 Progreso y Puntos

- El sistema registra el progreso del usuario por laboratorio: no iniciado, en progreso, o completado.
- Se requiere un mínimo del 60% de aciertos (3 de 5 preguntas) para que un envío cuente como aprobado.
- Al completar un laboratorio por primera vez con puntaje aprobatorio, el sistema suma automáticamente sus puntos al total del usuario mediante un trigger de base de datos.
- El usuario puede reintentar un laboratorio ya completado, pero los puntos solo se otorgan una vez.
- El sistema guarda el mejor puntaje (best_score_percent), el número de intentos (attempts_count) y el estado de cada laboratorio por usuario.

1.6 Ranking y Perfiles Públicos

- El sistema expone un ranking global de usuarios ordenado por puntos de mayor a menor.
- El ranking es paginado (20 usuarios por página por defecto).
- Cualquier visitante (sin sesión) puede ver el ranking y los perfiles públicos.
- El perfil público muestra username, bio, foto, puntos totales y laboratorios completados.

1.7 Foro Comunitario

- El sistema debe exponer un foro comunitario accesible públicamente para lectura.
- Solo los usuarios autenticados pueden publicar comentarios o respuestas.
- El foro soporta comentarios raíz y respuestas de un único nivel de anidamiento. No se permite responder a una respuesta.
- Los comentarios raíz se pagan (20 por página). Cada comentario raíz incluye el conteo de respuestas.
- El usuario puede eliminar sus propios comentarios. El administrador puede eliminar cualquier comentario.
- La eliminación de comentarios es un soft-delete: el contenido se reemplaza por '[comentario eliminado]' y el autor se muestra como null, pero el registro persiste en la base de datos.
- Los comentarios tienen un límite de 2000 caracteres.

1.8 Asistente Virtual (Uchi)

- La plataforma debe exponer un asistente de IA disponible en todas las páginas para usuarios autenticados.
- El asistente responde en tiempo real mediante streaming (Server-Sent Events).
- El asistente usa RAG (Retrieval-Augmented Generation) con TF-IDF sobre una base de conocimiento de la plataforma para inyectar contexto relevante antes de generar la respuesta.
- Solo se inyectan los FAQs con similitud coseno mayor o igual a 0.10 para no inflar innecesariamente el contexto del modelo.
- El asistente es un microservicio Python independiente del backend principal, desplegado por separado.

1.9 Administración

- El administrador puede crear, editar y publicar/despublicar cursos, módulos, laboratorios y preguntas.
- El administrador puede crear y editar actividades prácticas asociadas a preguntas de tipo actividad.
- El administrador puede agregar opciones de respuesta a preguntas de selección múltiple.
- El contenido no publicado no es visible para los usuarios regulares.
- Los administradores pueden acceder al contenido de laboratorios sin necesidad de estar inscritos en el curso.
- Los administradores pueden eliminar cualquier comentario del foro.

1.10 Estadísticas Públicas

- El sistema debe exponer un endpoint público con métricas globales de la plataforma: número de cursos, laboratorios, usuarios registrados y puntos totales acumulados.
- Estas métricas se muestran en la landing page para visitantes no autenticados.

2. Requerimientos No Funcionales

- Las contraseñas deben almacenarse hasheadas con bcrypt o argon2 (Bun.password nativo), nunca en texto plano.
- La foto de perfil solo acepta formato JPG/JPEG con un tamaño máximo de 5 MB. La imagen se almacena como bytes (BYTEA) en la base de datos, no como URL.
- El sistema debe responder las consultas del ranking en menos de 500 ms bajo carga normal, apoyado en el índice sobre users.points.
- La arquitectura del backend debe seguir el patrón Routes → Controllers → Services → DAO → Database.
- El contenido de los laboratorios se almacena en Markdown. El frontend es responsable de renderizarlo.
- Las respuestas generadas por las actividades deben ser únicas por usuario (hash HMAC-SHA256 con el JWT_SECRET como llave) para evitar que se compartan entre sí.
- La API debe validar que las 5 respuestas de un submission pertenezcan efectivamente a las preguntas del laboratorio enviado.
- El sistema debe soportar soft-delete en usuarios (campo deleted_at) para no perder historial de submissions y progreso.
- Los tokens de verificación de registro y de restablecimiento de contraseña se almacenan en memoria (Map en el servidor). Se pierden si el servidor reinicia.
- El envío de correos electrónicos debe realizarse mediante la Gmail REST API con OAuth2. No se permite SMTP, ya que Railway bloquea los puertos SMTP salientes.
- El sistema de puntuación debe estar garantizado a nivel de base de datos mediante triggers de PostgreSQL, independientemente de la capa de aplicación.
- Los slugs de cursos, módulos y laboratorios solo pueden contener letras minúsculas, números y guiones.
- Las respuestas de error de la API deben usar un formato JSON uniforme: { "error": "mensaje" }.
- El frontend debe ser una SPA desplegada en Vercel. El backend en Railway. El chatbot en Railway como servicio separado. La base de datos en Supabase (PostgreSQL).

- El sistema debe implementar el principio de mínimo privilegio en las proyecciones de datos: cada respuesta de la API expone únicamente los campos necesarios para el contexto (toPublic, toProfile, toAdmin, toSession, etc.).

3. Reglas de Negocio

1. Un usuario solo puede inscribirse una vez a un curso.
2. Un laboratorio tiene siempre exactamente 5 preguntas, ni más ni menos.
3. Una submission solo es válida si contiene respuestas para las 5 preguntas del laboratorio.
4. Los puntos de un laboratorio se suman al usuario únicamente la primera vez que lo completa con puntaje aprobatorio ($\geq 60\%$), sin importar cuántos intentos haga después.
5. Una pregunta de tipo actividad solo tiene una opción correcta: la respuesta generada por el sistema al completar exitosamente la actividad.
6. El usuario debe completar la actividad exitosamente antes de poder responder correctamente su pregunta asociada, ya que la respuesta correcta se genera en ese momento.
7. Una actividad está vinculada a exactamente una pregunta, y una pregunta solo puede tener como máximo una actividad.
8. El contenido de un curso, módulo o laboratorio despublicado no es accesible para usuarios regulares, aunque estén inscritos.
9. Un usuario con `deleted_at` no puede iniciar sesión ni aparecer en el ranking, pero su historial se conserva en la base de datos.
10. El score de una submission se calcula en el backend antes de persistirlo; el frontend nunca envía el puntaje calculado.
11. El foro solo soporta un nivel de respuestas: un comentario raíz puede tener respuestas, pero una respuesta no puede tener respuestas propias.
12. Al eliminar un usuario, los comentarios del foro que haya publicado persisten con contenido visible pero con `author = null` (SET NULL en la FK).
13. El hash de respuesta de una actividad es determinístico: para la misma combinación de usuario y actividad siempre se genera el mismo valor, usando HMAC-SHA256 con el `JWT_SECRET` como llave.
14. Un administrador no necesita estar inscrito en un curso para acceder al contenido de sus laboratorios.
15. El ranking excluye a los usuarios con soft-delete (`deleted_at IS NOT NULL`).